

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

TIER 1: CORE FUNDAMENTALS & LOCATORS

Assignment 3: Dynamic Dropdowns & Checkboxes

Name :	Ashmita Saha
Enrollment no. :	12023002028118
Department :	CSE (AI & ML)
Course :	Python for Automation
Academic Session :	2023-2027
Submission Date :	11 Sept, 2026

Module: Web Automation using Selenium WebDriver

Experiment Title: Automated Interaction with Dynamic Form Elements (Checkboxes & Auto-Suggest Dropdowns)

1. PROBLEM STATEMENT

Modern web applications frequently employ interactive elements such as dynamically generated checkboxes and auto-completing search dropdowns. Automated test scripts must reliably interact with these components despite unpredictable rendering times and dynamic DOM updates. This assignment requires developing an automated script to launch a target web application, programmatically select multiple interactive checkboxes, validate their active states using explicit assertions, and perform auto-suggest queries on a dynamic search input by dynamically filtering and selecting from a list of search results.

2. OBJECTIVE

- To master automated handling of interactive form controls such as dynamic checkboxes and multi-state input elements using Selenium WebDriver.
- To implement robust element selection and status verification using built-in conditional state methods like `is_selected()`.
- To handle dynamic search-as-you-type autocomplete dropdown fields by sending keystrokes, fetching option collections, and dynamically iterating through dynamic menus to select matching options.
- To enforce dynamic explicit wait strategies using `WebDriverWait` and standard `expected_conditions` to guarantee reliable synchronization without hardcoded delays.

3. TOOLS, SOFTWARE, AND CONCEPTS USED

- **Programming Language:** Python 3.x
- **Automation Library:** Selenium WebDriver
- **Browser Engine:** Google Chrome / ChromeDriver
- **Development Environment:** Visual Studio Code, PowerShell Terminal, Python Virtual Environment (`.venv`)
- **Core Concepts:**
 - *CSS Selectors & ID Locators:* Precise targeting of dynamic DOM elements.
 - *Explicit Synchronization:* Utilizing `WebDriverWait` alongside `expected_conditions` (e.g., `element_to_be_clickable`, `visibility_of_element_located`, `presence_of_all_elements_located`).
 - *Element Verification:* State checks using boolean conditions to validate input selections.

- *Dynamic Collection Iteration*: Fetching elements dynamically and matching string conditions at runtime.

4. SOURCE CODE OR IMPLEMENTATION STEPS

Implementation Steps:

1. Initialize Google Chrome browser instance using Selenium WebDriver.
2. Navigate to the web automation target URL (<https://rahulshettyacademy.com/AutomationPractice/>).
3. Instantiate an explicit wait object (WebDriverWait) with a timeout threshold of 10 seconds.
4. Iterate through checkbox values (option1, option2), wait until clickable, toggle selection, and assert state using `is_selected()`.
5. Locate the autocomplete text field, input search substring ("ger"), wait for suggested menu options to load, iterate through elements, and click the option matching "Germany".
6. Close browser instance cleanly using `driver.quit()`.

Python Source Code:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.get("https://rahulshettyacademy.com/AutomationPractice/")

wait = WebDriverWait(driver, 10)

# Iterate through target checkboxes and verify selection
for option in ["option1", "option2"]:
    checkbox = wait.until(EC.element_to_be_clickable(
        (By.CSS_SELECTOR, f"input[value='{option}']")))
    checkbox.click()
    assert checkbox.is_selected()

# Send search query keys to autocomplete input box
autocomplete = wait.until(
    EC.visibility_of_element_located((By.ID, "autocomplete")))
autocomplete.send_keys("ger")

# Wait for dynamically populated suggestions and pick target match
suggestions = wait.until(
    EC.presence_of_all_elements_located((By.CSS_SELECTOR, ".ui-menu-item")))

for item in suggestions:
    if item.text == "Germany":
        item.click()
        break
```

```
driver.quit()
```

5. OUTPUT WITH CLEARLY LABELLED SCREENSHOTS

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.get("https://rahulshettyacademy.com/AutomationPractice/")

wait = WebDriverWait(driver, 10)

for option in ["option1", "option2"]:
    checkbox = wait.until(EC.element_to_be_clickable(
        (By.CSS_SELECTOR, f"input[value='{option}']")))
    checkbox.click()
    assert checkbox.is_selected()

autocomplete = wait.until(EC.visibility_of_element_located...)
driver.quit()
```

Figure 1: Automation Script Source Code Implementation (Asst_3.py)

```
● PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium>Set-ExecutionPolicy -Scope Process...
● (.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> python Asst_3.py
Checkbox 1 selected: True
Checkbox 2 selected: True
Selected country: Germany
● (.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> █
```

Figure 2: Terminal Output demonstrating successful execution and dynamic assertions

6. BRIEF RESULT, OBSERVATION, AND CONCLUSION

Result:

The script executed successfully in the virtual environment (.venv). Both targeted checkboxes (Option 1 and Option 2) were located, toggled, and validated. The dynamic autocomplete field accepted the character sequence "ger", fetched dynamic drop-down suggestions, iterated through the dynamic list options, and successfully selected "Germany".

Observation:

- Explicit synchronization using `WebDriverWait` along with `expected_conditions` eliminated timing issues associated with dynamic DOM element rendering.
- Iterating over dynamically populated elements (such as dynamic suggestions matching `.ui-menu-item`) provided a resilient mechanism for element interaction without hardcoded static sleep delays.

Conclusion:

Automating dynamic web components requires flexible locator strategies and robust synchronization mechanisms. Through this assignment, standard techniques for dynamically handling checkboxes, assertions, search-as-you-type inputs, and option filtering in Selenium WebDriver were effectively demonstrated and validated.